

Simulation and Sensitivity Analysis of Chaotic Systems

Task 1 Report

Part A Discrete Chaotic Systems

A.2 – A.3 Logistic Map and Hénon Map

The logistic map, $x_{n+1} = r x_n(1 - x_n)$, was iterated at $r = 3.9$ from $x_0 = 0.5$ for 1000 iterations, discarding the first 300 as transient (Fig. 1). The Hénon map was iterated at $a = 1.4$, $b = 0.3$ from $(x_0, y_0) = (0, 0)$ for 5000 iterations, discarding the first 500 as transient (Fig. 2).

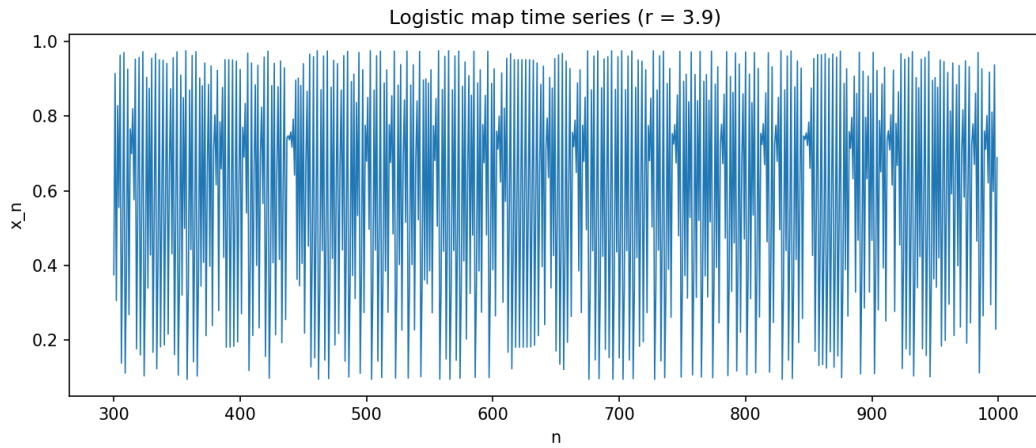


Figure 1: Logistic map time series at $r = 3.9$, after discarding transient.

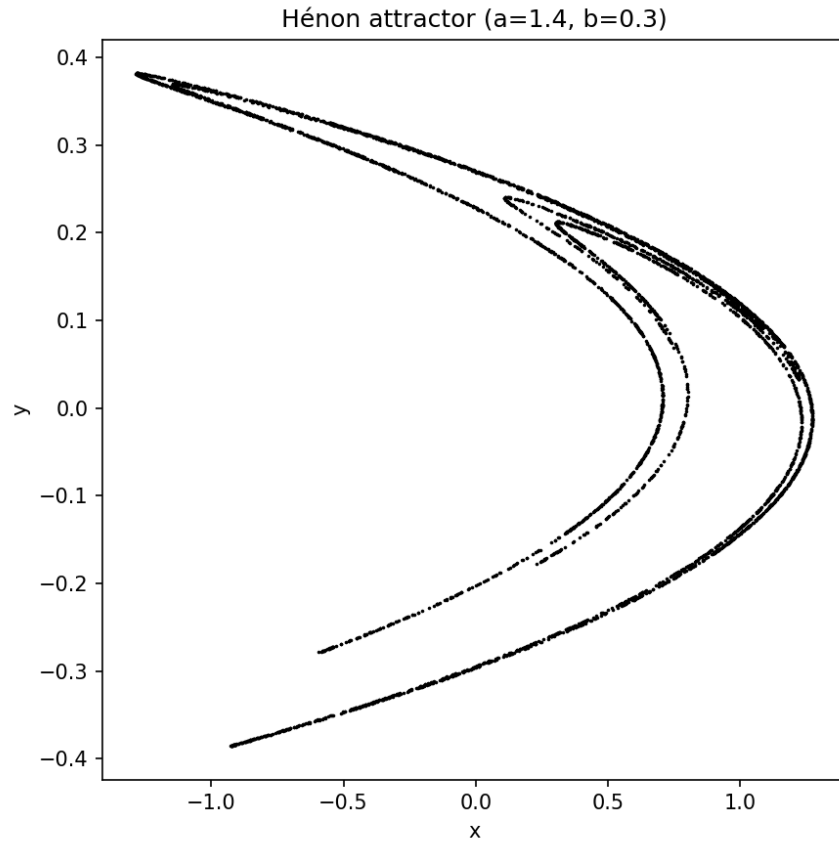


Figure 2: Hénon attractor ($a = 1.4$, $b = 0.3$): scatter of retained iterates.

A.4 Bifurcation Diagrams

Bifurcation diagrams were built by sweeping the control parameter, iterating each value many times, discarding a transient, and plotting the retained iterates against the parameter value: $r \in [2.5, 4.0]$ (1500 steps, 1000 iterations, transient 300) for the logistic map, and $a \in [1.0, 1.4]$ with $b = 0.3$ fixed (600 steps, 2000 iterations, transient 500) for the Hénon map.

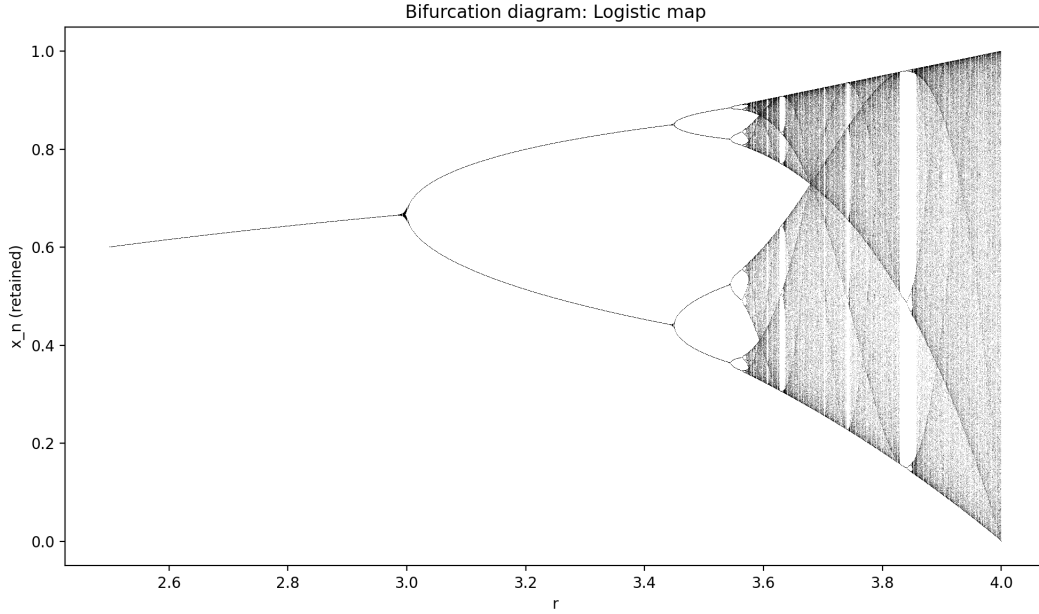


Figure 3: Bifurcation diagram of the logistic map: retained x_n vs. r .

This diagram shows a single fixed point up to $r \approx 3.0$, followed by a period doubling cascade ($2 \rightarrow 4 \rightarrow 8 \rightarrow \dots$) that accumulates into chaos beyond $r \approx 3.57$. Clearly visible periodic windows, such as near $r \approx 3.83$, briefly interrupt the chaotic bands before chaos resumes.

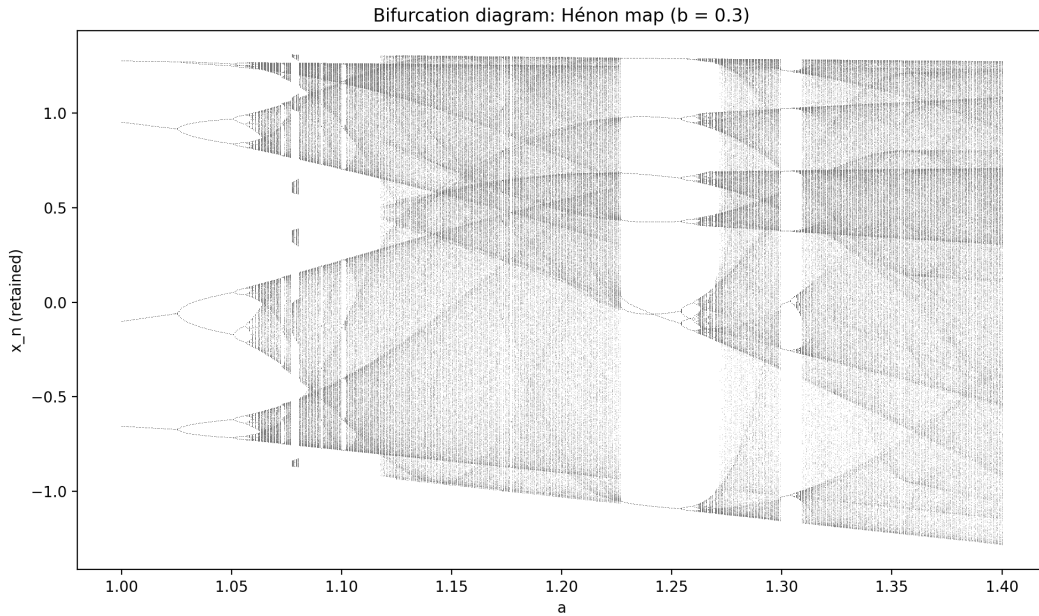


Figure 4: Bifurcation diagram of the Hénon map: retained x_n vs. a (with $b = 0.3$).

This diagram shows the same qualitative period doubling route to chaos but for a two dimensional map, so the branching structure is denser and more tangled. Sharp periodic windows are interspersed throughout the chaotic region, reflecting the additional folding and stretching introduced by the second state variable.

Part B Continuous Chaotic Systems

All continuous systems below were integrated with a fixed step RK4 scheme implemented from scratch.

B.2 Lorenz System

$$\begin{aligned}\dot{x} &= \sigma(y - x) \\ \dot{y} &= x(\rho - z) - y \\ \dot{z} &= xy - \beta z\end{aligned}$$

Table 1: Settings: Lorenz system

Setting	Value
State variables	(x, y, z)
Parameters	$\sigma = 10, \rho = 28, \beta = 8/3$
Initial condition	$(x_0, y_0, z_0) = (1, 1, 1)$
Time step, dt	0.01
Simulation length	$T = 50$ (≈ 5000 steps)
Solver	RK4 (fixed step, custom implementation)

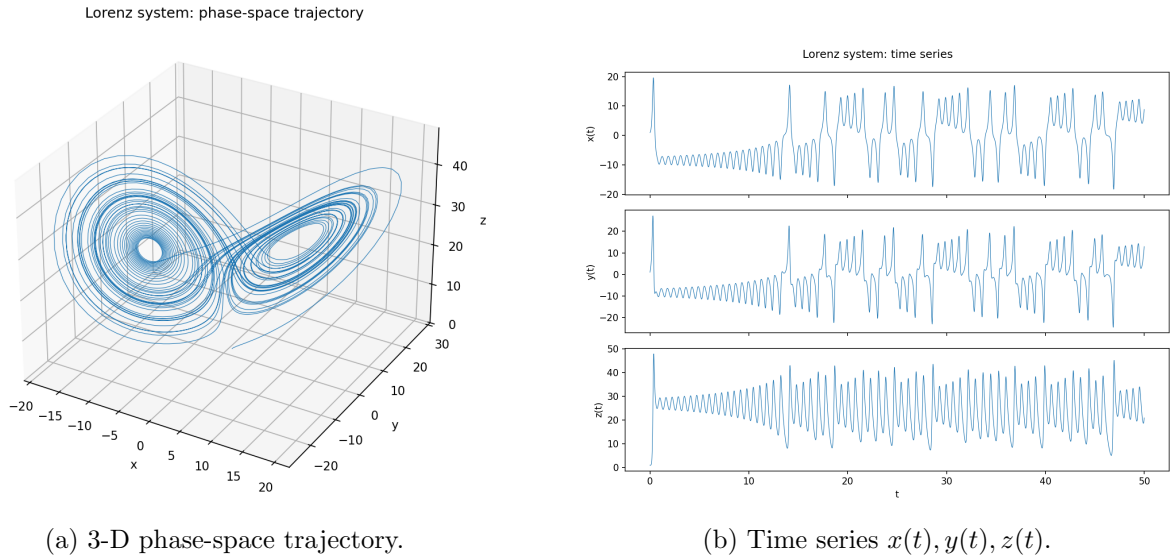


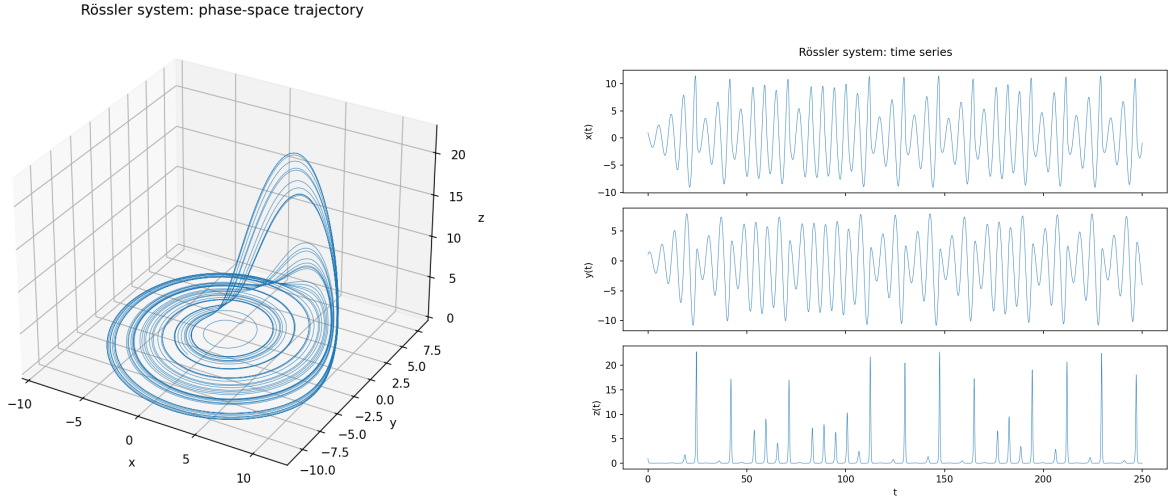
Figure 5: Lorenz system at classic chaotic parameters.

B.3 Rössler System

$$\begin{aligned}\dot{x} &= -y - z \\ \dot{y} &= x + ay \\ \dot{z} &= b + z(x - c)\end{aligned}$$

Table 2: Settings: Rössler system

Setting	Value
State variables	(x, y, z)
Parameters	$a = 0.2, b = 0.2, c = 5.7$
Initial condition	$(x_0, y_0, z_0) = (1, 1, 1)$
Time step, dt	0.01
Simulation length	$T = 250$
Solver	RK4 (fixed step, custom implementation)



(a) 3-D phase-space trajectory.

(b) Time series $x(t), y(t), z(t)$.

Figure 6: Rössler system at classic chaotic parameters.

B.4 Chen System

$$\begin{aligned}
 \dot{x} &= a(y - x) \\
 \dot{y} &= (c - a)x - xz + cy \\
 \dot{z} &= xy - bz
 \end{aligned}$$

Table 3: Settings: Chen system

Setting	Value
State variables	(x, y, z)
Parameters	$a = 35, b = 3, c = 28$
Initial condition	$(x_0, y_0, z_0) = (-0.1, 0.5, -0.6)$
Time step, dt	0.002
Simulation length	$T = 50$
Solver	RK4 (fixed step, custom implementation)

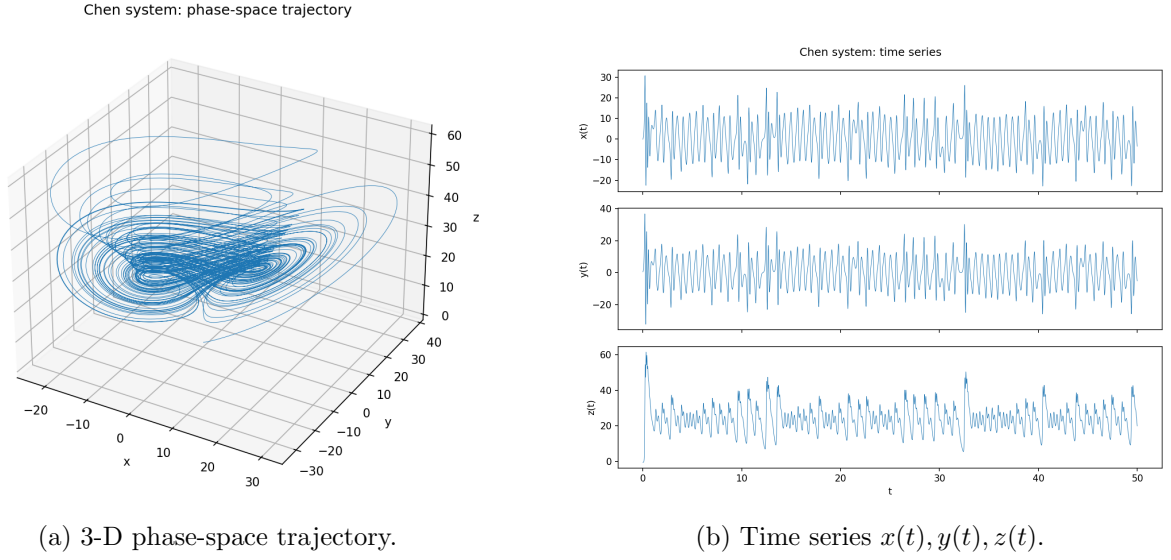


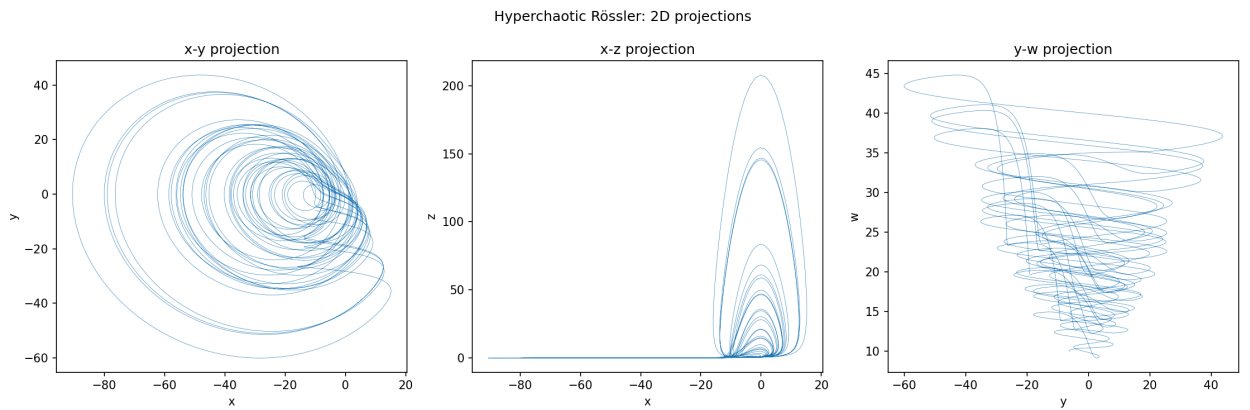
Figure 7: Chen system at classic chaotic parameters.

B.5 Hyperchaotic Rössler System

$$\begin{aligned}\dot{x} &= -y - z \\ \dot{y} &= x + ay + w \\ \dot{z} &= b + xz \\ \dot{w} &= -cz + dw\end{aligned}$$

Table 4: Settings: Hyperchaotic Rössler system

Setting	Value
State variables	(x, y, z, w)
Parameters	$a = 0.25, b = 3, c = 0.5, d = 0.05$
Initial condition	$(x_0, y_0, z_0, w_0) = (-10, -6, 0, 10)$
Time step, dt	0.01
Simulation length	$T = 250$
Solver	RK4 (fixed step, custom implementation)

Figure 8: Hyperchaotic Rössler: 2-D projections of the 4-D phase space $(x-y, x-z, y-w)$.

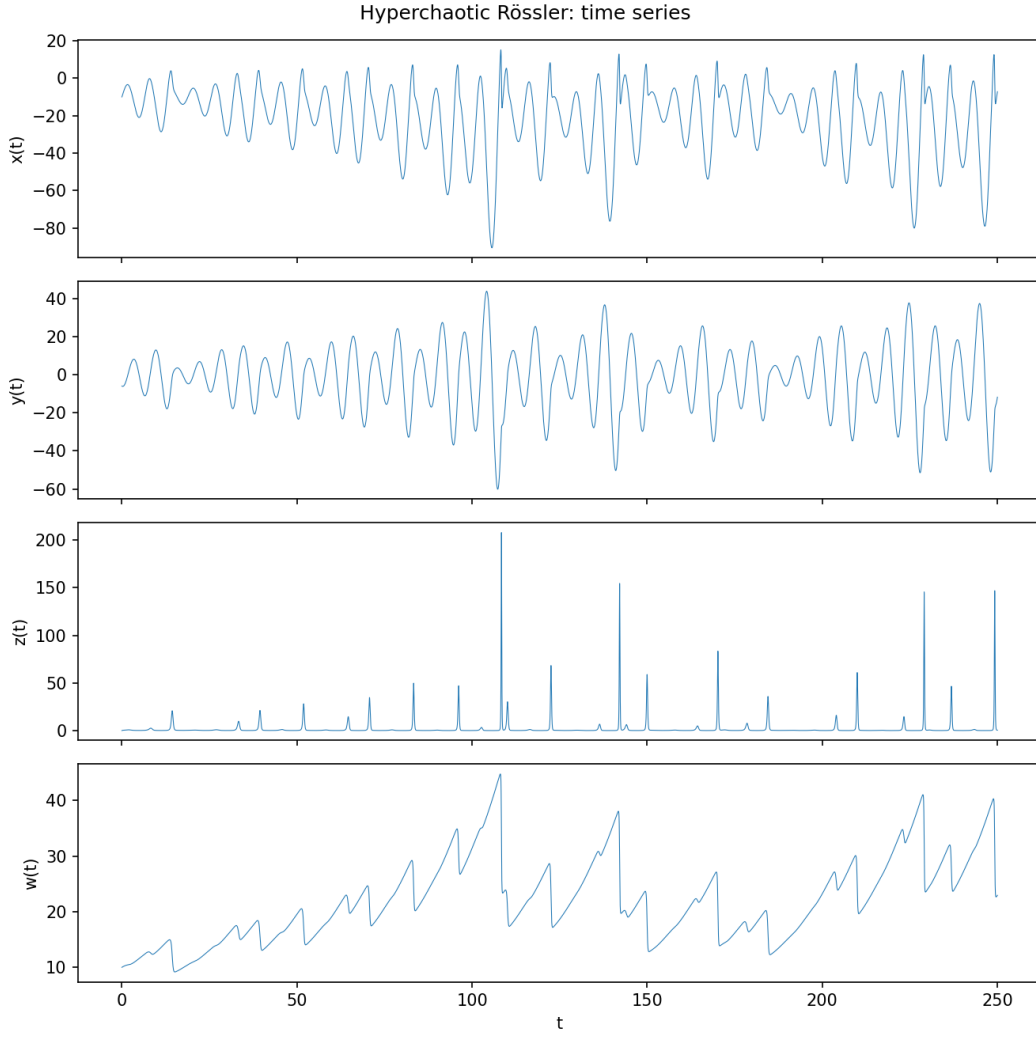


Figure 9: Hyperchaotic Rössler: time series of all four states.

Part C Sensitivity Analysis

C.1 Parameter Sweep of the Lorenz System

With $\sigma = 10$ and $\beta = 8/3$ fixed, ρ was swept over $[0, 30]$ in steps of 0.25. At each value the system was integrated with the same IC, dt , and solver as B.2, the first 20 time units were discarded as transient, and the local maxima of $z(t)$ were recorded.

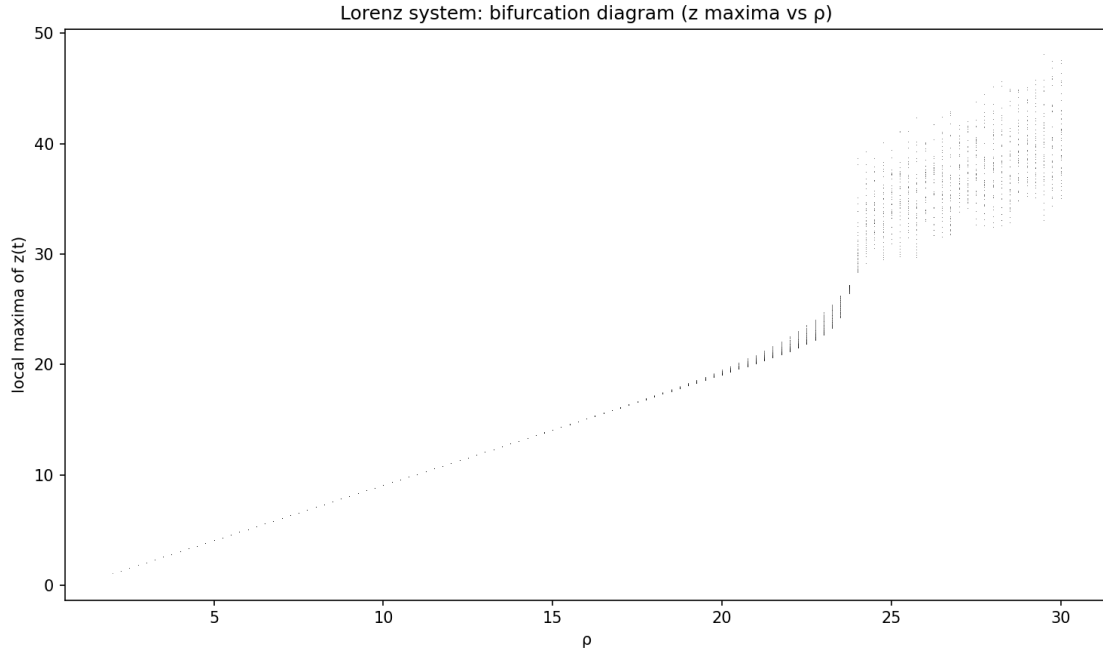


Figure 10: Lorenz bifurcation-style diagram: local maxima of $z(t)$ vs. ρ .

For $\rho \lesssim 24$ the diagram shows a single, smoothly rising branch of z maxima, meaning the trajectory settles onto a fixed point or simple periodic orbit at each of these values. Beyond $\rho \approx 24$ to 28 the single branch fans out abruptly into a dense vertical spread of maxima, marking the onset of chaos. This transition is the continuous system analogue of the period doubling cascade seen in Part A, though it appears here as a sudden widening rather than a resolved doubling sequence, consistent with the known onset of Lorenz chaos near $\rho \approx 24.74$.

C.2 Step-Size Sensitivity

The Lorenz system was re-simulated at the standard chaotic parameters using three step sizes ($dt = 0.001, 0.01, 0.05$), with identical IC, solver, and simulation length ($T = 50$).

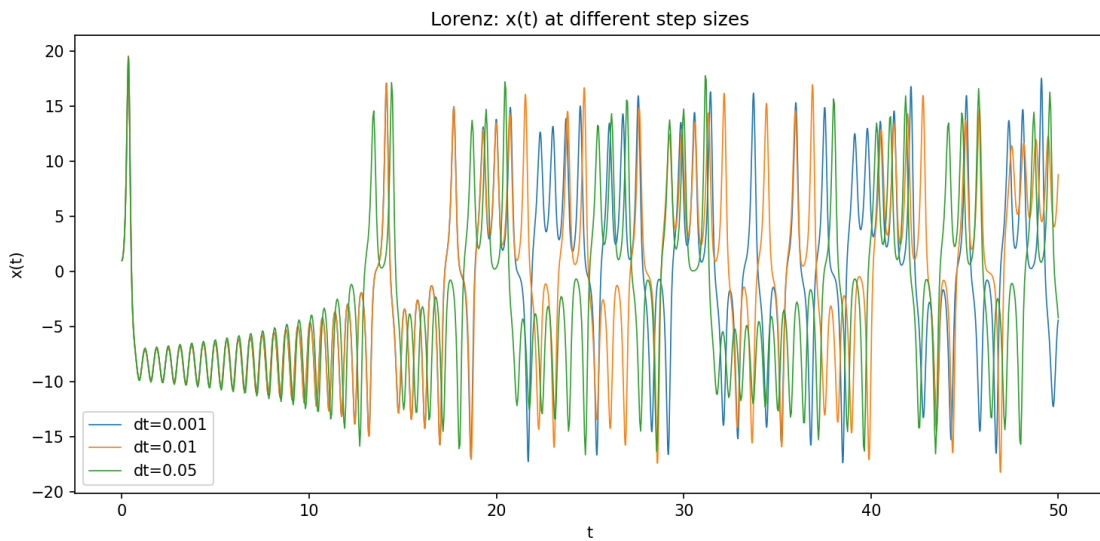


Figure 11: Overlay of $x(t)$ for the three step sizes.

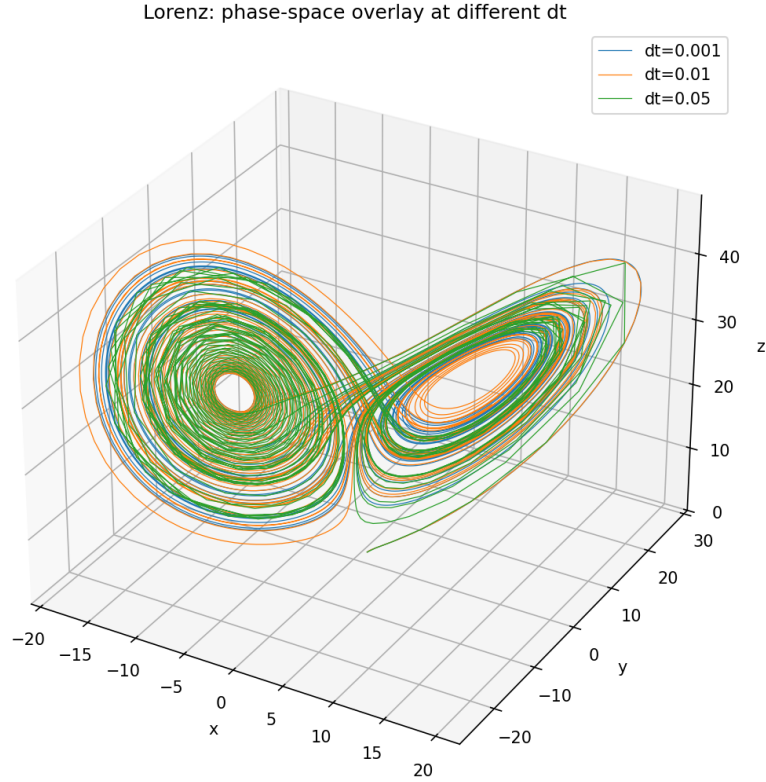


Figure 12: Overlay of the 3-D phase-space trajectories for the three step sizes.

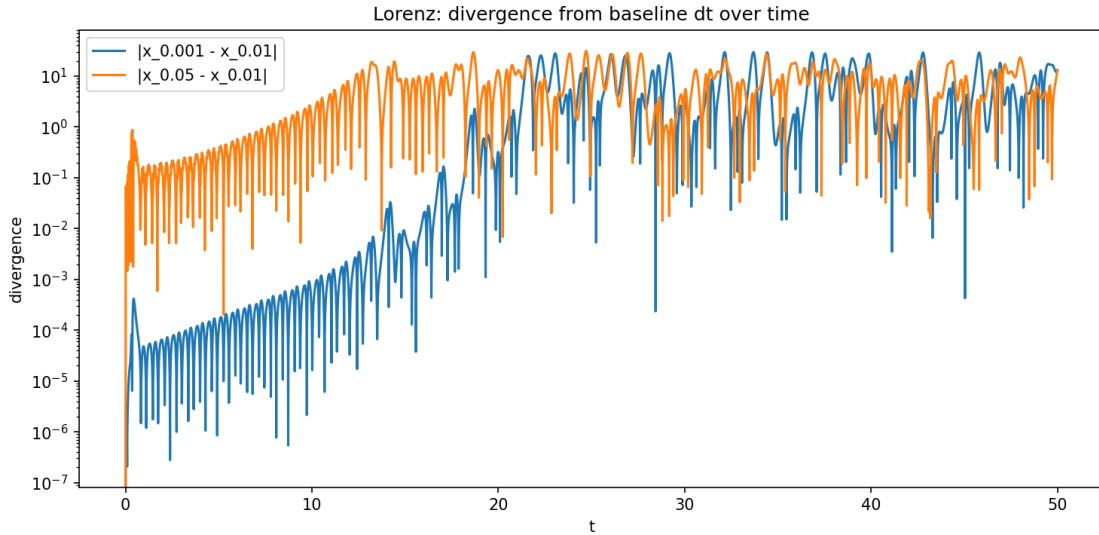


Figure 13: Pairwise divergence $|x_{dt}(t) - x_{0.01}(t)|$ vs. t (log scale), relative to the $dt = 0.01$ baseline.

All three step sizes agree closely at the start of the simulation, and the divergence plot confirms this, since both the $dt = 0.001$ and $dt = 0.05$ trajectories track the baseline with small oscillating error for roughly the first 10 to 15 time units. Because the Lorenz system is chaotic, this small initial numerical error is exponentially amplified by sensitive dependence on initial conditions, so the divergence for both step sizes grows by several orders of magnitude and saturates around $t \approx 20$ to 25, after which the trajectories are effectively decorrelated. The finer step ($dt = 0.001$) diverges roughly 2 to 3 orders of magnitude more slowly than the coarser step ($dt = 0.05$) due to its smaller local truncation error, though it too eventually diverges since no finite step size can indefinitely

track a chaotic trajectory. Qualitatively, $dt = 0.01$ and $dt = 0.001$ reproduce the same well formed butterfly attractor, while $dt = 0.05$ is visibly coarser with minor deviations, indicating $dt = 0.01$ is an adequate and efficient choice for this system.

Conclusion

Comparing discrete and continuous chaos, both settings show the same route from stable behaviour to periodicity to full chaos, but the route looks different in each. The discrete maps in Part A show it as a clear period doubling cascade, while the continuous ρ sweep in Part C.1 shows a more sudden fan out once the critical parameter is crossed, with no clearly resolved doubling sequence at this resolution. Discrete maps are also cheaper to compute since they are simple recursive formulas with no integration error, whereas continuous systems in Part B need a solver and step size that directly affect accuracy, as shown by the step-size sensitivity analysis in Part C.2.

Comparing chaotic and hyperchaotic behaviour, the ordinary chaotic systems (Lorenz, Rössler and Chen) each settle into one clear attractor shape, either a double scroll or a single scroll spiral, with a single direction of exponential divergence. The hyperchaotic Rössler system needs a fourth state variable and has two directions of divergence at once, giving denser, higher amplitude, less predictable trajectories than any 3-D system here. So hyperchaos is not just more chaos; it is a distinct behaviour that needs the extra dimension to appear.

Appendix: Simulation Code

Part A

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # ----- Logistic map: time series -----
5
6 def logistic_map(r, x0, n_iter):
7     x = np.empty(n_iter)
8     x[0] = x0
9     for i in range(1, n_iter):
10         x[i] = r * x[i - 1] * (1 - x[i - 1])
11     return x
12
13 r_chaotic = 3.9
14 x0 = 0.5
15 n_iter = 1000
16 transient = 300
17
18 x_series = logistic_map(r_chaotic, x0, n_iter)
19 n = np.arange(n_iter)
20
21 plt.figure(figsize=(9, 4))
22 plt.plot(n[transient:], x_series[transient:], lw=0.8)
23 plt.xlabel("n")
24 plt.ylabel("x_n")
25 plt.title(f"Logistic map time series (r = {r_chaotic})")
26 plt.tight_layout()
27 plt.savefig("logistic_timeseries.png", dpi=150)
28 plt.show()
29
30 # ----- Henon map: attractor -----
31
32 def henon_map(a, b, x0, y0, n_iter):
33     x = np.empty(n_iter)
34     y = np.empty(n_iter)

```

```

36     x[0], y[0] = x0, y0
37     for i in range(1, n_iter):
38         x[i] = 1 - a * x[i - 1] ** 2 + y[i - 1]
39         y[i] = b * x[i - 1]
40     return x, y
41
42 a, b = 1.4, 0.3
43 hx0, hy0 = 0.0, 0.0
44 n_iter_h = 5000
45 transient_h = 500
46
47 x_h, y_h = henon_map(a, b, hx0, hy0, n_iter_h)
48
49 plt.figure(figsize=(6, 6))
50 plt.scatter(x_h[transient_h:], y_h[transient_h:], s=0.5, color="black")
51 plt.xlabel("x")
52 plt.ylabel("y")
53 plt.title(f"Hnon attractor (a={a}, b={b})")
54 plt.tight_layout()
55 plt.savefig("henon_attractor.png", dpi=150)
56 plt.show()
57
58 # ----- Bifurcation diagram: logistic map -----
59
60
61 n_r = 1500
62 r_vals = np.linspace(2.5, 4.0, n_r)
63 x = np.full(n_r, 0.5)
64
65 n_iter_bif = 1000
66 transient_bif = 300
67
68 r_plot, x_plot = [], []
69
70 for i in range(n_iter_bif):
71     x = r_vals * x * (1 - x)
72     if i >= transient_bif:
73         r_plot.append(r_vals)
74         x_plot.append(x.copy())
75
76 r_plot = np.concatenate(r_plot)
77 x_plot = np.concatenate(x_plot)
78
79 plt.figure(figsize=(10, 6))
80 plt.plot(r_plot, x_plot, ',k', alpha=0.25)
81 plt.xlabel("r")
82 plt.ylabel("x_n (retained)")
83 plt.title("Bifurcation diagram: Logistic map")
84 plt.tight_layout()
85 plt.savefig("logistic_bifurcation.png", dpi=200)
86 plt.show()
87
88 # ----- Bifurcation diagram: Hnon map -----
89
90
91 n_a = 600
92 a_vals = np.linspace(1.0, 1.4, n_a)
93 b_fixed = 0.3
94 xa = np.zeros(n_a)
95 ya = np.zeros(n_a)
96
97 n_iter_bif_h = 2000
98 transient_bif_h = 500
99
100 a_plot, xa_plot = [], []

```

```

101
102 for i in range(n_iter_bif_h):
103     xa_new = 1 - a_vals * xa ** 2 + ya
104     ya_new = b_fixed * xa
105     xa, ya = xa_new, ya_new
106     if i >= transient_bif_h:
107         a_plot.append(a_vals)
108         xa_plot.append(xa.copy())
109
110 a_plot = np.concatenate(a_plot)
111 xa_plot = np.concatenate(xa_plot)
112
113 plt.figure(figsize=(10, 6))
114 plt.plot(a_plot, xa_plot, ',k', alpha=0.25)
115 plt.xlabel("a")
116 plt.ylabel("x_n (retained)")
117 plt.title("Bifurcation diagram: Hnon map (b = 0.3)")
118 plt.tight_layout()
119 plt.savefig("henon_bifurcation.png", dpi=200)
120 plt.show()

```

Listing 1: Part A: Logistic map, Hénon map, and their bifurcation diagrams.

Part B

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4
5 def rk4_integrate(deriv, state0, dt, T):
6     n_steps = int(T / dt)
7     state0 = np.array(state0, dtype=float)
8     dim = len(state0)
9     traj = np.empty((n_steps + 1, dim))
10    traj[0] = state0
11    t = np.linspace(0, n_steps * dt, n_steps + 1)
12
13    x = state0.copy()
14    for i in range(n_steps):
15        k1 = deriv(x)
16        k2 = deriv(x + 0.5 * dt * k1)
17        k3 = deriv(x + 0.5 * dt * k2)
18        k4 = deriv(x + dt * k3)
19        x = x + (dt / 6.0) * (k1 + 2*k2 + 2*k3 + k4)
20        traj[i + 1] = x
21
22    return t, traj
23
24
25 # ----- B.2 Lorenz system -----
26
27 def lorenz_deriv(state, sigma=10.0, rho=28.0, beta=8/3):
28     x, y, z = state
29     dx = sigma * (y - x)
30     dy = x * (rho - z) - y
31     dz = x * y - beta * z
32     return np.array([dx, dy, dz])
33
34 t_lorenz, traj_lorenz = rk4_integrate(lorenz_deriv, [1, 1, 1], dt=0.01, T=50)
35
36 fig = plt.figure(figsize=(7, 6))
37 ax = fig.add_subplot(projection='3d')
38 ax.plot(traj_lorenz[:, 0], traj_lorenz[:, 1], traj_lorenz[:, 2], lw=0.5)
39 ax.set_xlabel("x"); ax.set_ylabel("y"); ax.set_zlabel("z")

```

```

40 ax.set_title("Lorenz system: phase-space trajectory")
41 plt.tight_layout()
42 plt.savefig("lorenz_3d.png", dpi=150)
43 plt.show()
44
45 fig, axs = plt.subplots(3, 1, figsize=(9, 7), sharex=True)
46 labels = ["x(t)", "y(t)", "z(t)"]
47 for i in range(3):
48     axs[i].plot(t_lorenz, traj_lorenz[:, i], lw=0.6)
49     axs[i].set_ylabel(labels[i])
50 axs[-1].set_xlabel("t")
51 fig.suptitle("Lorenz system: time series")
52 plt.tight_layout()
53 plt.savefig("lorenz_timeseries.png", dpi=150)
54 plt.show()
55
56
57 # ----- B.3 Rssler system -----
58
59 def rossler_deriv(state, a=0.2, b=0.2, c=5.7):
60     x, y, z = state
61     dx = -y - z
62     dy = x + a * y
63     dz = b + z * (x - c)
64     return np.array([dx, dy, dz])
65
66 t_rossler, traj_rossler = rk4_integrate(rossler_deriv, [1, 1, 1], dt=0.01, T=250)
67
68 fig = plt.figure(figsize=(7, 6))
69 ax = fig.add_subplot(projection='3d')
70 ax.plot(traj_rossler[:, 0], traj_rossler[:, 1], traj_rossler[:, 2], lw=0.4)
71 ax.set_xlabel("x"); ax.set_ylabel("y"); ax.set_zlabel("z")
72 ax.set_title("Rssler system: phase-space trajectory")
73 plt.tight_layout()
74 plt.savefig("rossler_3d.png", dpi=150)
75 plt.show()
76
77 fig, axs = plt.subplots(3, 1, figsize=(9, 7), sharex=True)
78 for i in range(3):
79     axs[i].plot(t_rossler, traj_rossler[:, i], lw=0.5)
80     axs[i].set_ylabel(labels[i])
81 axs[-1].set_xlabel("t")
82 fig.suptitle("Rssler system: time series")
83 plt.tight_layout()
84 plt.savefig("rossler_timeseries.png", dpi=150)
85 plt.show()
86
87
88 # ----- B.4 Chen system -----
89
90 def chen_deriv(state, a=35.0, b=3.0, c=28.0):
91     x, y, z = state
92     dx = a * (y - x)
93     dy = (c - a) * x - x * z + c * y
94     dz = x * y - b * z
95     return np.array([dx, dy, dz])
96
97 t_chen, traj_chen = rk4_integrate(chen_deriv, [-0.1, 0.5, -0.6], dt=0.002, T=50)
98
99 fig = plt.figure(figsize=(7, 6))
100 ax = fig.add_subplot(projection='3d')
101 ax.plot(traj_chen[:, 0], traj_chen[:, 1], traj_chen[:, 2], lw=0.4)
102 ax.set_xlabel("x"); ax.set_ylabel("y"); ax.set_zlabel("z")
103 ax.set_title("Chen system: phase-space trajectory")
104 plt.tight_layout()

```

```

105 plt.savefig("chen_3d.png", dpi=150)
106 plt.show()
107
108 fig, axs = plt.subplots(3, 1, figsize=(9, 7), sharex=True)
109 for i in range(3):
110     axs[i].plot(t_chen, traj_chen[:, i], lw=0.5)
111     axs[i].set_ylabel(labels[i])
112 axs[-1].set_xlabel("t")
113 fig.suptitle("Chen system: time series")
114 plt.tight_layout()
115 plt.savefig("chen_timeseries.png", dpi=150)
116 plt.show()
117
118 # ----- B.5 Hyperchaotic Rossler system -----
119
120
121 def hyperchaotic_rossler_deriv(state, a=0.25, b=3.0, c=0.5, d=0.05):
122     x, y, z, w = state
123     dx = -y - z
124     dy = x + a * y + w
125     dz = b + x * z
126     dw = -c * z + d * w
127     return np.array([dx, dy, dz, dw])
128
129 t_hr, traj_hr = rk4_integrate(hyperchaotic_rossler_deriv, [-10, -6, 0, 10], dt=0.01, T=250)
130
131 fig, axs = plt.subplots(1, 3, figsize=(15, 5))
132 projections = [(0, 1, "x", "y"), (0, 2, "x", "z"), (1, 3, "y", "w")]
133 for ax, (i, j, xl, yl) in zip(axs, projections):
134     ax.plot(traj_hr[:, i], traj_hr[:, j], lw=0.3)
135     ax.set_xlabel(xl); ax.set_ylabel(yl)
136     ax.set_title(f"{xl}-{yl} projection")
137 fig.suptitle("Hyperchaotic Rossler: 2D projections")
138 plt.tight_layout()
139 plt.savefig("hyperchaotic_rossler_projections.png", dpi=150)
140 plt.show()
141
142 fig, axs = plt.subplots(4, 1, figsize=(9, 9), sharex=True)
143 labels_hr = ["x(t)", "y(t)", "z(t)", "w(t)"]
144 for i in range(4):
145     axs[i].plot(t_hr, traj_hr[:, i], lw=0.5)
146     axs[i].set_ylabel(labels_hr[i])
147 axs[-1].set_xlabel("t")
148 fig.suptitle("Hyperchaotic Rossler: time series")
149 plt.tight_layout()
150 plt.savefig("hyperchaotic_rossler_timeseries.png", dpi=150)
151 plt.show()

```

Listing 2: Part B: RK4 integrator and Lorenz, Rössler, Chen, and hyperchaotic Rössler systems.

Part C

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4
5 def rk4_integrate(deriv, state0, dt, T):
6     n_steps = int(T / dt)
7     state0 = np.array(state0, dtype=float)
8     dim = len(state0)
9     traj = np.empty((n_steps + 1, dim))
10    traj[0] = state0
11    t = np.linspace(0, n_steps * dt, n_steps + 1)
12

```

```

13     x = state0.copy()
14     for i in range(n_steps):
15         k1 = deriv(x)
16         k2 = deriv(x + 0.5 * dt * k1)
17         k3 = deriv(x + 0.5 * dt * k2)
18         k4 = deriv(x + dt * k3)
19         x = x + (dt / 6.0) * (k1 + 2*k2 + 2*k3 + k4)
20         traj[i + 1] = x
21
22     return t, traj
23
24
25 def lorenz_deriv(state, sigma=10.0, rho=28.0, beta=8/3):
26     x, y, z = state
27     dx = sigma * (y - x)
28     dy = x * (rho - z) - y
29     dz = x * y - beta * z
30     return np.array([dx, dy, dz])
31
32
33 # ===== C.1 rho-sweep (bifurcation-style plot) =====
34
35 sigma, beta = 10.0, 8/3
36 IC = [1, 1, 1]
37 dt_sweep = 0.01
38 T_sweep = 50          # same length as B.2
39 transient_cutoff = 20  # discard first 20 time units
40
41 rho_values = np.arange(0, 30.01, 0.25)  # within doc's 0.1-0.5 step range
42 rho_list = []
43 z_maxima = []
44
45 for rho in rho_values:
46     deriv = lambda state, rho=rho: lorenz_deriv(state, sigma=sigma, rho=rho, beta=beta)
47     t, traj = rk4_integrate(deriv, IC, dt=dt_sweep, T=T_sweep)
48     z = traj[:, 2]
49
50     mask = t > transient_cutoff
51     z_kept = z[mask]
52
53     # local maxima: bigger than both neighbours
54     is_peak = (z_kept[1:-1] > z_kept[:-2]) & (z_kept[1:-1] > z_kept[2:])
55     peaks = z_kept[1:-1][is_peak]
56
57     rho_list.extend([rho] * len(peaks))
58     z_maxima.extend(peaks)
59
60 plt.figure(figsize=(10, 6))
61 plt.plot(rho_list, z_maxima, ',k', alpha=0.5)
62 plt.xlabel("")
63 plt.ylabel("local maxima of z(t)")
64 plt.title("Lorenz system: bifurcation diagram (z maxima vs )")
65 plt.tight_layout()
66 plt.savefig("lorenz_rho_sweep.png", dpi=150)
67 plt.show()
68
69
70 # ===== C.2 step-size sensitivity =====
71
72 IC = [1, 1, 1]
73 T = 50
74 dts = [0.001, 0.01, 0.05]
75 results = {}
76
77 for dt in dts:

```

```

78     t, traj = rk4_integrate(lambda s: lorenz_deriv(s), IC, dt=dt, T=T)
79     results[dt] = (t, traj)
80
81 # (i) overlay  $x(t)$ 
82 plt.figure(figsize=(10, 5))
83 for dt in dts:
84     t, traj = results[dt]
85     plt.plot(t, traj[:, 0], label=f"dt={dt}", lw=0.8)
86 plt.xlabel("t"); plt.ylabel("x(t)")
87 plt.title("Lorenz: x(t) at different step sizes")
88 plt.legend()
89 plt.tight_layout()
90 plt.savefig("stepsize_xt_overlay.png", dpi=150)
91 plt.show()
92
93 # (ii) overlay 3-D phase-space trajectories
94 fig = plt.figure(figsize=(8, 7))
95 ax = fig.add_subplot(projection='3d')
96 for dt in dts:
97     t, traj = results[dt]
98     ax.plot(traj[:, 0], traj[:, 1], traj[:, 2], label=f"dt={dt}", lw=0.6)
99 ax.set_xlabel("x"); ax.set_ylabel("y"); ax.set_zlabel("z")
100 ax.set_title("Lorenz: phase-space overlay at different dt")
101 ax.legend()
102 plt.tight_layout()
103 plt.savefig("stepsize_3d_overlay.png", dpi=150)
104 plt.show()
105
106 # (iii) divergence  $|x_{dt}(t) - x_{baseline}(t)|$  vs  $t$ 
107 baseline_dt = 0.01
108 t_base, traj_base = results[baseline_dt]
109 x_base = traj_base[:, 0]
110
111 plt.figure(figsize=(10, 5))
112 for dt in dts:
113     if dt == baseline_dt:
114         continue
115     t_dt, traj_dt = results[dt]
116     x_interp = np.interp(t_base, t_dt, traj_dt[:, 0])
117     divergence = np.abs(x_interp - x_base)
118     plt.plot(t_base, divergence, label=f"|x_{dt} - x_{baseline_dt}|")
119 plt.xlabel("t"); plt.ylabel("divergence")
120 plt.yscale("log")
121 plt.title("Lorenz: divergence from baseline dt over time")
122 plt.legend()
123 plt.tight_layout()
124 plt.savefig("stepsize_divergence.png", dpi=150)
125 plt.show()

```

Listing 3: Part C: Lorenz ρ -sweep and step-size sensitivity analysis.